Task 4 Vision

October 24, 2026

Assignment Goal

The goal of this task is to teach you how to build, train, and test neural networks with PyTorch. After completing this assignment you will know how to:

- Write the code to create, train, and test a basic neural network from scratch using PyTorch
- Load in datasets and train a neural network with a dataloader
- Learn about loss functions, optimizers, and gradient descent in PyTorch
- Understand the general scaffolding behind training neural networks in PyTorch

Assignment Specification

This assignment is composed of 4 files:

- **main.py** Main orchestration
- **train.py** Training and evaluation functions
- **data.py** Data loading and pre-processing
- **model.py** Model architecture

We will provide tests to ensure that your functions produce the proper outputs; however, not all functions can be tested, so you should rely on this document for specifications.

Why PyTorch?

You could, in principle, write a neural network using nothing but NumPy: store the weights as arrays, multiply them together to get predictions, and update them by hand. People did exactly this for years. The problem is that the hard

part of training is not the forward pass but computing *gradients*: how much each weight should change to reduce the loss. For a network with millions of weights, deriving and coding those derivatives by hand is slow, error-prone, and has to be redone every time you change the architecture. PyTorch removes this burden and several others.

Automatic differentiation (autograd). Every tensor in PyTorch can record the operations performed on it. When you compute a loss from your model's output, PyTorch has quietly built a graph of every step that led there. Calling `loss.backward()` walks that graph in reverse and applies the chain rule automatically, storing the gradient of the loss with respect to every weight in that weight's `.grad` attribute. For example:

```
w = torch.tensor(3.0, requires_grad=True)
loss = (2 * w - 4) ** 2
loss.backward()
print(w.grad)   # tensor(8.)  since d/dw (2w-4)^2 = 4(2w-4) = 8 at w=3
```

The same single line, `loss.backward()`, works whether your model has one weight or a billion, and whether it is the small network in this assignment or a large language model. You write only the forward pass; PyTorch handles the calculus.

Modularity. Because every model is an `nn.Module`, models are built out of smaller modules the same way programs are built out of functions. A linear layer is a module, an activation is a module, your whole network is a module, and your network could itself be used as a single layer inside an even bigger network. `nn.Module` automatically finds every learnable parameter inside every sub-module, so `model.parameters()` hands the complete list to the optimizer no matter how deeply the pieces are nested. This makes it easy to swap components in and out: changing `nn.ReLU()` to `nn.Tanh()`, or adding another hidden layer, is a one-line change, and the training code does not need to change at all.

Many things included. PyTorch comes with the standard building blocks of deep learning already implemented, tested, and optimized: layers (`nn.Linear`, `nn.Conv2d`, ...), loss functions (`nn.CrossEntropyLoss`, `nn.MSELoss`, ...), optimizers (`torch.optim.SGD`, `torch.optim.Adam`, ...), and data utilities (`Dataset`, `DataLoader`). You will use all of these in the rest of this assignment. Libraries such as `torchvision` add common datasets and pretrained models on top.

GPU acceleration. Neural network training is dominated by large matrix multiplications, which GPUs perform far faster than CPUs. In PyTorch, moving a model or tensor to a GPU is a single call, `.to("cuda")`, and the rest of your code stays the same.

It is just Python. PyTorch builds its computation graph on the fly as your code runs, rather than requiring you to define the whole graph up front. This means you can use ordinary Python `if` statements and `for` loops inside a model, insert `print` statements to inspect tensor shapes mid-computation, and step through your model with a normal debugger. This flexibility is a large part of why PyTorch has become the most widely used framework in deep learning research.

Files

Since each file may (or may not) depend on the completion of other files, please work on these files in the order of the following sections.

1 model.py

Background

In most deep learning codebases, the neural network architectures live in their own file(s), separate from the data loading and training code. Here we use `model.py`. If a project contains several architectures, it is common to give each one its own file named after the model, such as `resnet.py` or `mobilenet.py`.

PyTorch builds models using classes. Every model is a subclass of `torch.nn.Module`. By inheriting from it, your model automatically gets useful behavior: PyTorch tracks all of its learnable parameters (so an optimizer can update them), lets you move it to a GPU with `.to(device)`, and lets you switch between training and evaluation modes with `.train()` and `.eval()`.

Every `nn.Module` needs two methods:

- `__init__`: defines the layers of the network (and therefore its weights).
- `forward`: defines how an input flows through those layers to produce an output.

The architecture

You will build a simple fully connected network with one hidden layer, often called a multilayer perceptron (MLP). Its layers, in order, are:

1. `nn.Flatten()`: reshapes each image from a 2D grid of pixels (for example $1 \times 28 \times 28$) into a single vector of length `input_dim` (for example 784). Linear layers expect vectors, not grids.
2. `nn.Linear(input_dim, hidden_dim)`: the hidden layer.
3. `nn.ReLU()`: a nonlinear activation, $\mathrm{ReLU}(z) = \max(0, z)$. Without it, two stacked linear layers would collapse into a single linear layer, and the network could only learn linear functions.

4. `nn.Linear(hidden_dim, num_classes)`: the output layer, which produces one score per class.

The **hidden dimension** is the number of neurons in the hidden layer. It is a *hyperparameter*: a value you choose rather than one the network learns. A larger hidden dimension gives the network more capacity to learn complex patterns, but it also makes the model slower to train and more prone to overfitting. You will choose this value in `main.py`.

Note that the network does *not* end with a softmax. It returns raw scores, called **logits**. The loss function we use in `train.py` (`nn.CrossEntropyLoss`) applies softmax internally, so adding one here would be redundant and hurt training.

Your task

1. Import PyTorch and its neural network module: `import torch` and `import torch.nn as nn`.
2. Create a class called `Model` that inherits from `nn.Module`.
3. Write its `__init__` method. It should take three arguments: `input_dim`, `hidden_dim`, and `num_classes`. (For our dataset `num_classes` is always 10 and `input_dim` is $1 \cdot 28 \cdot 28$, but keep both as parameters so the model can be reused.) Inside `__init__`:
   (a) Call `super().__init__()` as the very first line. This sets up the internals of `nn.Module`; without it, PyTorch will raise an error when you try to add layers.
   (b) Save the three arguments as `self.input_dim`, `self.hidden_dim`, and `self.num_classes`.
   (c) Create `self.network` as an `nn.Sequential` containing the four layers listed above, in order. `nn.Sequential` chains layers together so that the output of each one is fed into the next.
4. Write a `forward` method that takes a single argument `x` (a batch of inputs) and returns `self.network(x)`.

When you are done, your class should have exactly these attributes, since the tests check for them by name: `self.input_dim`, `self.hidden_dim`, `self.num_classes`, and `self.network`.

Checking your shapes

It helps to know what shapes to expect. If `x` is a batch of 64 images of size $1 \times 28 \times 28$, then:

After layer	Shape
Input	`(64, 1, 28, 28)`
Flatten	`(64, 784)`
Linear (hidden)	`(64, hidden_dim)`
ReLU	`(64, hidden_dim)`
Linear (output)	`(64, 10)`

Notice that the first dimension (the batch size) never changes. Every layer operates on each example in the batch independently.

You never call `forward` directly. Instead, call the model like a function, `model(x)`, and PyTorch will call `forward` for you.

Testing

Once you have finished, run `python3 model.py` to check that your model is built correctly.

2 data.py

Background

It's rather hard to train a model without data, so importing it correctly is incredibly important. PyTorch splits this job into two pieces:

- A `Dataset` knows how many examples there are (`len(dataset)`) and how to fetch a single one (`dataset[i]`), returning an `(image, label)` pair. It answers the question "what is example i?"
- A `DataLoader` wraps a `Dataset` and hands examples to your training loop in *batches*. It answers the question "in what order, and how many at a time?"

Why batches? Computing the gradient on the entire dataset at once is accurate but slow and memory-hungry; computing it on one example at a time is fast but very noisy and iterating over the full dataset is extremely slow. Batches sit in between: their gradient is a reasonable estimate of the true one, and a GPU can process the whole batch in parallel. This is why the first dimension of every tensor in the previous section was the batch size. It is convention to use powers of 2 for the batch size like 32, 64, 128, 256, 512, 1024, ...

Why shuffle? If the training data were always seen in the same order (for example, all the 0s, then all the 1s), each batch would contain only one class and the model would chase whichever class it saw most recently. Shuffling the training set at the start of every epoch makes each batch a random sample.

Train and test splits. We always evaluate on data the model has never trained on. A model that scores well on its training data but poorly on new data has *memorized* rather than *learned*; this is called **overfitting**. The test set is our only honest estimate of how the model will do in the real world, so it must never be used to update the weights.

Your task

1. Import the MNIST dataset class from TorchVision: `from torchvision.datasets import MNIST`
2. Import TorchVision's transforms module: `from torchvision import transforms`
3. Import PyTorch's `DataLoader` class: `from torch.utils.data import DataLoader`
4. Define a function called `get_mnist(batch_size)`. It takes one argument, `batch_size`, and returns a train dataloader and a test dataloader.
5. Inside the function, define the transformation that will be applied to every image:
   (a) Use `transforms.Compose([...])` to chain the following two steps together.
   (b) `transforms.ToTensor()` converts each image from a PIL image to a PyTorch tensor and scales pixel values from $[0, 255]$ to $[0, 1]$.
   (c) `transforms.Normalize((0.1307,), (0.3081,))` normalizes each image using the mean and standard deviation of the MNIST training set.
   (d) Store the result in a variable called `transform`.
6. Create the train and test datasets by calling `MNIST()` with these arguments:
   (a) `root="./data"`: the folder where the data will be stored.
   (b) `train=True` for `train_dataset`, and `train=False` for `test_dataset`.
   (c) `download=True`: downloads the data if it is not already present.
   (d) `transform=transform`: applies the transformation defined above.
7. Create the dataloaders by calling `DataLoader()`:
   (a) `train_loader`: pass `train_dataset`, set `batch_size=batch_size`, and set `shuffle=True`.
   (b) `test_loader`: pass `test_dataset`, set `batch_size=batch_size`, and set `shuffle=False`. We do not shuffle test data because evaluation does not depend on order.
8. Return both dataloaders: `return train_loader, test_loader`

Testing

Once you have finished, run `python3 data.py` to check that your dataloaders are built correctly. The first run downloads MNIST into `./data`, which may take a minute.

3 train.py

Background

Training a neural network is an optimization problem where you find the weights that make the loss as small as possible. Three parts are necessary:

The loss function. A loss function measures how wrong the model's predictions are, as a single number. For classification we use **cross-entropy loss** (`nn.CrossEntropyLoss`). It converts the logits to probabilities with a softmax and then takes the negative log of the probability assigned to the correct class. If the model is confident and right, the loss is near 0; if it is confident and wrong, the loss is large.

Gradient descent. The gradient of the loss with respect to a weight tells us which direction increases the loss, so we step each weight a small amount in the opposite direction:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

where η is the **learning rate**, another hyperparameter. Too small and training crawls; too large and the loss bounces around or diverges. Because we compute the gradient on a mini-batch rather than the whole dataset, this is called *stochastic* gradient descent (SGD).

The optimizer. An optimizer object (`torch.optim.SGD`, `torch.optim.Adam`, ...) holds a reference to the model's parameters and applies the update rule above when you call `optimizer.step()`. Different optimizers use different update rules, but they all share the same interface.

The training loop. Every PyTorch training loop, from this assignment to large research models, follows a similar five steps for each batch:

1. `optimizer.zero_grad()`: clear the old gradients. PyTorch *adds* new gradients to whatever is already stored in `.grad`, so forgetting this step silently mixes gradients from previous batches into the current one.
2. `outputs = model(inputs)`: the forward pass.
3. `loss = loss_fn(outputs, labels)`: measure the error.
4. `loss.backward()`: autograd computes every gradient.

5. `optimizer.step()`: update the weights.

One full pass through the training set is called an **epoch**.

Training mode vs. evaluation mode. Some layers (such as dropout and batch normalization) behave differently during training and evaluation. `model.train()` and `model.eval()` switch between the two. Our model has no such layers, but calling them is good practice. During evaluation we also wrap the loop in `with torch.no_grad():`. This tells autograd not to record the computation graph, which saves memory and time since we will never call `backward()` on test data.

Devices. The model and the data must live on the same device. The model is moved once (in `main.py`), but each batch comes off the `DataLoader` on the CPU, so inside the loop you must move it with `inputs.to(device)` and `labels.to(device)`.

Your task

1. Import PyTorch: `import torch`
2. Define a function called `train_one_epoch(model, train_loader, loss_fn, optimizer, device)`. It trains the model for one epoch and returns the average training loss.
   (a) Call `model.train()` to put the model in training mode.
   (b) Create a variable `total_loss = 0.0` to accumulate the loss across batches.
   (c) Loop over the batches with `for inputs, labels in train_loader:`
   (d) Inside the loop, move the batch to the correct device: `inputs = inputs.to(device)` and `labels = labels.to(device)`.
   (e) Perform the five steps of the training loop:
      i. Clear old gradients: `optimizer.zero_grad()`
      ii. Forward pass: `outputs = model(inputs)`
      iii. Compute the loss: `loss = loss_fn(outputs, labels)`
      iv. Backward pass: `loss.backward()`
      v. Update the weights: `optimizer.step()`
   (f) Add the batch loss to the running total: `total_loss += loss.item()`. Using `.item()` converts the loss tensor to a plain Python number, so we don't keep the computation graph in memory.
   (g) After the loop, return the average loss per batch: `return total_loss / len(train_loader)`
3. Define a function called `evaluate(model, test_loader, loss_fn, device)`. It measures the model's performance on the test set and returns the average test loss and the accuracy.

(a) Call `model.eval()` to put the model in evaluation mode.

(b) Create three variables: `total_loss = 0.0`, `correct = 0`, and `total = 0`.

(c) Wrap the entire loop in `with torch.no_grad():` so that autograd does not record the computation graph.

(d) Loop over the batches with `for inputs, labels in test_loader:`

(e) Move the batch to the correct device, exactly as in training.

(f) Run the forward pass and compute the loss: `outputs = model(inputs)` and `loss = loss_fn(outputs, labels)`. Do *not* call `zero_grad()`, `backward()`, or `step()`; we are only measuring, not learning.

(g) Add the batch loss to the total: `total_loss += loss.item()`

(h) Get the predicted class for each image by taking the index of the largest logit: `preds = outputs.argmax(dim=1)`

(i) Count the correct predictions and the number of images in the batch: `correct += (preds == labels).sum().item()` and `total += labels.size(0)`

(j) After the loop, compute and return the results:

  i. Average loss: `avg_loss = total_loss / len(test_loader)`

  ii. Accuracy: `accuracy = correct / total`

  iii. `return avg_loss, accuracy`

Note. You do not create the loss function or the optimizer in this file. They are created in `main.py` and passed in as arguments, for example `loss_fn = nn.CrossEntropyLoss()` and `optimizer = torch.optim.SGD(model.parameters(), lr=0.01)`.

Testing

Once you have finished, run `python3 train.py` to check that your training and evaluation functions run correctly.

4 main.py

Background

`main.py` is the entry point. It does very little work itself; its job is to choose the hyperparameters and wire the other three files together. Keeping orchestration separate from the pieces it orchestrates means you can run a new experiment by changing a few numbers here without touching the model, data, or training code.

Reproducibility. Weight initialization and data shuffling are random, so two runs of the same code can give slightly different results. Calling `torch.manual_seed` at the start fixes the random number generator so that results are repeatable, which makes debugging and comparing experiments much easier.

The `if __name__ == "__main__":` guard. Code under this guard runs only when the file is executed directly (`python3 main.py`), not when it is imported by another file such as a test. Your top-level logic lives in a `main()` function, which is called from under the guard.

Your task

You should now be able to use the files you have built to put everything together. The starter code already reads the command-line arguments, defines `main(num_epochs, batch_size, learning_rate=0.01)`, calls it from under the `if __name__ == "__main__":` guard, and sets `device` for you inside `main()`. Fill in the body of `main()` so that it does the following.

1. **Set up.**
   (a) Fix the random seed with `torch.manual_seed(0)`.
   (b) Use the hyperparameters passed in as arguments: `num_epochs` and `batch_size` come from the `--epochs` and `--batch_size` flags (defaults 5 and 64), and `learning_rate` defaults to 0.01. Do not hard-code these values inside `main()`, or the flags will be ignored.
2. **Create the model.** Create your model with the correct input size, a hidden dimension of 128, and the correct number of classes, then move it to the device with `.to(device)`. *Hint:* each MNIST image is 28×28 pixels, and there are 10 digits.
3. **Load the data.** Call `get_mnist()` with your batch size to get the train and test dataloaders.
4. **Create the loss function and optimizer.** Use `nn.CrossEntropyLoss()` for the loss, and an optimizer such as `torch.optim.SGD`. Remember that the optimizer needs the model's parameters and the learning rate.
5. **Train and log.** Loop over the number of epochs. In each epoch:
   (a) Train the model for one epoch with `train_one_epoch()`.
   (b) Measure the train and test accuracy with `evaluate()`. Call it twice: once with `train_loader` and once with `test_loader`. Even though its parameter is named `test_loader`, `evaluate()` works on any dataloader.
   (c) Print the epoch number, train accuracy, and test accuracy, for example: `Epoch 1:  train acc = 0.912, test acc = 0.918`

Running the code. To run this code you should run
`python3 main.py --epochs N --batch_size N`
we will use this command to evaluate your code so make sure everything is wired up with the starter code we give you.